

MedGuard AI – Member Task Distribution Document

Phase 1 – Requirement & Planning (All Members – 1 Week)

🎯 Goal:

Freeze architecture, tech stack, and module responsibilities.

👤 Backend Lead

- Finalize API structure
- Define endpoint contracts (request/response format)
- Decide authentication flow (JWT)
- Define error response standards
- Plan mobile-compatible API responses

🤖 AI/ML Lead

- Finalize embedding model (Sentence Transformers)
- Decide FAISS index type
- Research & select Ollama LLM model
- Define RAG similarity strategy

🗄️ DB & DevOps Lead

- Design initial DB schema (3NF)
- Define ENUM types:
 - Legality Status
 - Result Status
- Plan Docker container architecture
- Define persistent storage plan (FAISS + DB)

🎨 Frontend Lead (Web + Mobile)

- Create UI wireframes (Web + Mobile)
- Define user flow:
 - Medicine verification
 - Symptom module
- Define mobile app architecture:
 - Flutter → API Gateway

✅ Deliverable:

- Architecture Diagram
- ER Diagram
- Service Flow Design (Web + Mobile)

Phase 2 – Database & Dataset Setup

 DB & DevOps Lead (Primary Owner)

- Create PostgreSQL schema:
 - Users
 - Medicines
 - Medicine Barcodes
 - Embedding Metadata
 - Verification History
 - Symptom History
- Add indexing for performance
- Setup migrations
- Configure local Docker DB

 AI/ML Lead

- Clean and normalize medicine dataset
- Assign legality ENUM values
- Validate barcode mappings
- Prepare embedding text chunks

 Backend Lead

- Connect Django to PostgreSQL
- Create models & serializers
- Implement migrations
- Setup JWT authentication

 Frontend Lead

- Build Login/Register UI (Web + Mobile)
- Implement authentication integration

 Deliverable:

- Working PostgreSQL DB
- Cleaned Dataset
- Authentication functional

Phase 3 – RAG & FAISS Setup (AI Core Phase)

 AI/ML Lead (Primary Owner)

- Generate embeddings
- Build FAISS index
- Implement similarity search
- Tune similarity threshold
- Ensure <5 sec response time

 DB & DevOps Lead

- Setup FAISS persistent Docker volume

- Store embedding metadata mapping
- Ensure restart-safe index

Backend Lead

- Create RAG search API endpoint
- Integrate FAISS with Django service
- Return similarity score

Frontend Lead

- Create medicine text input UI (Web + Mobile)
- Display similarity score
- Handle loading states

Deliverable:

- Fully working RAG Search API
- Fast semantic search implemented

Phase 4 – Medicine Verification Service

Backend Lead (Primary Owner)

- OCR integration (Google Vision)
- Barcode decoding (pyzbar)
- MFG/EXP extraction (Regex)
- Expiry validation logic
- Legality classification
- Authenticity decision logic
- ENUM handling
- Store verification history

AI/ML Lead

- Validate similarity scoring
- Assist in authenticity logic tuning
- Optimize embedding retrieval

DB & DevOps Lead

- Implement verification logging table
- Optimize indexing
- Monitor query performance

Frontend Lead

- Multi-modal upload UI:
 - Text
 - Image
 - Barcode
- Display color-coded result:

- Green – Authentic
- Red – Fake
- Orange – Expired
- Purple – Illegal

✓ Deliverable:

- Complete Medicine Verification Module (Web + Mobile)

Phase 5 – Symptom Remedy LLM Module

 AI/ML Lead (Primary Owner)

- Ollama setup
- LLM model selection
- Strict prompt engineering
- Enforce mandatory disclaimer
- Prevent:
 - Diagnosis
 - Prescription drugs
 - Emergency advice

 Backend Lead

- Create symptom API endpoint
- Handle LLM timeout & error control
- Implement safety response filtering

 DB & DevOps Lead

- Optional symptom logging table
- Monitor RAM & CPU usage
- Resource optimization

 Frontend Lead

- Symptom input UI (Web + Mobile)
- Display:
 - Remedy
 - Proportions
 - Precautions
 - Disclaimer (mandatory)

✓ Deliverable:

- Safe & Functional Symptom LLM Module

Phase 6 – Dockerization & Cloud Deployment

 DB & DevOps Lead (Primary Owner)

- Write Dockerfiles

- Setup Docker Compose:
 - api-gateway
 - medicine-service
 - symptom-llm-service
 - postgres-db
 - ollama
- Configure persistent FAISS storage
- Deploy to AWS/DigitalOcean
- Setup HTTPS & firewall
- Configure backups

Backend Lead

- Production configs
- Secure JWT handling
- Environment variables setup

AI/ML Lead

- Optimize FAISS memory usage
- Tune embedding dimension
- Optimize LLM response speed

Frontend Lead

- Production build (Web)
- Generate signed Android APK
- Connect mobile app to live APIs

Deliverable:

- Fully Dockerized Cloud-Ready System
- Live Web Platform
- Android App Connected

Phase 7 – Testing & Optimization

Backend Lead

- API testing
- Error handling validation
- Security validation
- JWT expiry testing

AI/ML Lead

- Similarity threshold tuning
- LLM hallucination checks
- Performance measurement

DB & DevOps Lead

- Load testing (100 users)
- Backup verification
- Monitoring setup

 Frontend Lead

- UX testing
- Responsive design validation
- Android compatibility testing
- Camera permission testing
- Web-Mobile sync validation

 Deliverable:

- Stable Production Version
- Performance & Security Verified

Clear Responsibility Matrix (Final Version)

Module	Primary Owner	Support
API Gateway	Backend ▾	DevOps ▾
PostgreSQL Design	DB Lead ▾	Backend ▾
RAG + FAISS	AI Lead ▾	Backend ▾
OCR + Barcode	Backend ▾	AI ▾
Authenticity Logic	Backend ▾	AI ▾
Symptom LLM	AI Lead ▾	Backend ▾
Docker Deployment	DevOps ▾	Backend ▾
Web UI	Frontend ▾	Backend ▾
Mobile App (Flutter)	Frontend ▾	Backend ▾
Cloud Infrastructure	DevOps ▾	Backend ▾

Final Role Summary

Person	Role	Main Responsibility
Person 1	Backend Lead	API Gateway, Verification Logic, Symptom API, Security
Person 2	AI/ML Lead	Dataset, Embeddings, FAISS, RAG, Ollama Integration
Person 3	DB & DevOps Lead	PostgreSQL, Docker, FAISS

		Persistence, Deployment
Person 4	Frontend Lead	Web UI + Flutter Mobile App + API Integration